

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Coding Challenge Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Coding Challenge Task
Weightage:	40% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	Yuvasri.B	Reg. No.:	192421231
Section / Batch:	BTech IT	Date:	19/08/26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Write clear code for the given problem.
- Analyze the Time complexity and Space complexity of your code using dynamic programming and greedy strategies

Question Type	Focus Area	Questions
Coding Challenge Task	Focus on Code and analyze optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Coding Challenge Task

Code of Conduct

I Yuvasri .B/ 192421231 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: yuvasri

Q40. Maximum Length Chain of Pairs

Given pairs of numbers where each pair has a start and end value, the goal is to form the longest chain such that the end of one pair is less than the start of the next. Efficient selection is required to maximize chain length. Use Greedy or Dynamic Programming techniques.

Input Format:

n = number of pairs pairs[][]

Sample Input:

n = 3

(5,24) (15,25) (27,40) Expected

Output:

Max Chain Length = 2

INTRODUCTION

The **Maximum Length Chain of Pairs** problem finds the largest number of pairs that can be arranged in a chain. A pair (a, b) can be followed by another pair (c, d) only when $b < c$. We use the **Greedy Algorithm** by sorting the pairs based on their ending values and selecting the pair with the smallest possible end value. This leaves more space for selecting the remaining pairs.

AIM

To find the maximum length chain of pairs using the **Greedy algorithm**, where the end value of one pair is less than the start value of the next pair.

PSEUDOCODE

Algorithm MaximumChain(pairs, n)

1. Sort all pairs according to their ending value.
2. Set count = 1
3. Set lastEnd = pairs[0].end
4. For i = 1 to n-1:

If pairs[i].start > lastEnd:

count = count + 1

lastEnd = pairs[i].end

5. Return count End Algorithm

PYTHON CODE

```
n = int(input("Enter number of pairs: "))

pairs = []

for i in range(n):

    start, end = map(int, input(f"Enter pair {i + 1}: ").split())

    pairs.append((start, end))

# Sort pairs by ending value
pairs.sort(key=lambda x: x[1])

count = 0

last_end = float('-inf')

chain = []

for start, end in pairs:

    if start > last_end:

        chain.append((start, end))

        count += 1

        last_end = end

print("\nSorted Pairs:", pairs)

print("Selected Chain:", chain)

print("Max Chain Length =", count)
```

DAA MINI PROJECT · Q40

Maximum Length Chain of Pairs

Visualize the Greedy Algorithm Step by Step. Discover how the Greedy approach efficiently selects compatible pairs to form the longest possible chain.

Explore Algorithm →

Try Visualizer

Live concept preview · earliest finish wins

(5, 24) → (15, 25) → (27, 40)

Selected pairs preserve the most space for what comes next.

Problem Statement

Given a set of pairs (a, b), form the longest possible chain such that the second value of one pair is less than the first value of the next pair.

$$\text{previousEnd} < \text{currentStart}$$

For example, (5, 24) → (27, 40) works because $24 < 27$.

Worked Example

Input: (5,24), (15,25), (27,40)

(5, 24) ✓ Selected

(15, 25) ✗ Rejected · $15 > 24$ is false(27, 40) ✓ Selected · $27 > 24$

Maximum Chain Length = 2

Greedy Strategy

Select the pair with the smallest ending value first. This leaves maximum possible space for selecting future pairs.

Input

↓

Sort by End Value

↓

Select Earliest Ending Pair

↓

Check Compatibility

↓

Build Chain

↓

Maximum Chain

Pseudocode

```
sort pairs by ending value

lastEnd = -infinity
count = 0

for each pair:
    if pair.start > lastEnd:
        select pair
        count++
        lastEnd = pair.end

return count
```

Complexity

TIME
COMPLEXITY

$O(n \log n)$
Sorting requires $O(n \log n)$.

SPACE
COMPLEXITY

$O(n)$
Pairs and selected results are stored.

Choosing the pair that ends earliest gives the greatest opportunity to add more compatible pairs later.

Try the Algorithm →

Enter any valid pairs, then move through the sort and selection process one decision at a time.

Enter Your Pairs

Start must be less than end. Negative values are supported.

Number of pairs

Generate

Pair 1 - Start

End

Pair 2 - Start

End

Pair 3 - Start

End

Find Maximum Chain →

Load Sample Data

Ready to run

Step 1 of 4

← Previous

Next →

Auto Run

Pause

Step 1 - Original Pairs

INPUT

(5, 24)

(15, 25)

(27, 40)

Step 2 - Sorted Pairs

BY END VALUE

Pairs are sorted according to their ending values.

(5, 24)

→

(15, 25)

→

(27, 40)

Step 3 - Selection Process

START > LAST END

Checking (5, 24)
Condition: 5 > -∞

Checking...

Checking (15, 25)
Condition: 15 > 24

Waiting

Checking (27, 40)
Condition: 27 > 24

Waiting



CHAINIFY

MAXIMUM LENGTH CHAIN OF PAIRS

HomeAlgorithmVisualizerAnalysis

RESULTS DESK · 04

Result & Analysis

A concise dashboard for explaining the outcome, performance, and trade-offs of the greedy solution.

MAXIMUM CHAIN LENGTH

(5, 24) → (27, 40)

2

Selected pairs from your latest visualizer run.

PERFORMANCE PROFILE

Algorithm Performance

Worked Example

Input: (5, 24), (15, 25), (27, 40)

(5, 24)	✓ Selected
(15, 25)	✗ Rejected
(27, 40)	✓ Selected

Final: (5, 24) → (27, 40)

Max Chain Length = 2

Comparison

APPROACH	TIME	IDEA
Greedy	$O(n \log n)$	Select earliest ending pair
Dynamic Programming	$O(n^2)$	Compare possible chains

Greedy is the efficient standard approach because the earliest finish leaves room for the most future pairs.

Advantages of Greedy

- ✓ Simple implementation
- ✓ Efficient sorting-based solution
- ✓ Easy to visualize

Limitations

The solution depends on the specific structure of this problem. Not every optimization problem can be solved by repeatedly making a locally optimal choice.

Run Another Example →

Applications of Maximum Length Chain of Pairs

The Maximum Length Chain of Pairs problem is useful in many real-world scheduling and optimization applications.

- **Job Scheduling:** Selecting the maximum number of non-overlapping jobs.
- **Activity Selection:** Choosing activities that do not conflict with each other.
- **Event Planning:** Selecting the maximum number of events that can be attended without time overlap.
- **Task Scheduling:** Arranging tasks efficiently based on their start and end times.
- **Resource Management:** Selecting activities that use resources for non-overlapping time periods.

The problem demonstrates how a **Greedy Algorithm** can make efficient decisions by selecting the pair with the earliest ending value.

Conclusion

The Maximum Length Chain of Pairs problem can be efficiently solved using the **Greedy approach**. By sorting the pairs based on their ending values and selecting compatible pairs, we can obtain the maximum chain length efficiently. For the given example, the chain is **(5,24) → (27,40)**, giving a maximum chain length of **2**. The algorithm has a time complexity of **$O(n \log n)$** due to sorting, making it suitable for large inputs.

RUBRICS FOR CALCULATION

Coding Challenge Task

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				
Algorithm	25%				
Code Implementation	20%				
Competitive Performance	20%				
Test Case Handling	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____